

# Design Pattern

When you develop an application you come across many problems.

Before also many developers faced the same problem.

So when they came across such problems, then they devised some solutions.

We also use their solution or wisdom to solve our problems as well.

These solutions are called Design patterns.

flexible Design -

when any new feature comes then it should take minimum code changes & minimum time.

To use such design we use certain features like OOPS, SOLID and design pattern.

Remember: "Change is the only Constant"

↓  
Design pattern is a solution that

Design your application such that new feature takes minimal code changes & minimum time.

There are almost 23 design patterns and all says the same thing:-

In your application there are 2 parts

Things that don't get change, remain constant

↓  
Static Part

Things that keep on changing

↓  
Dynamic Part.

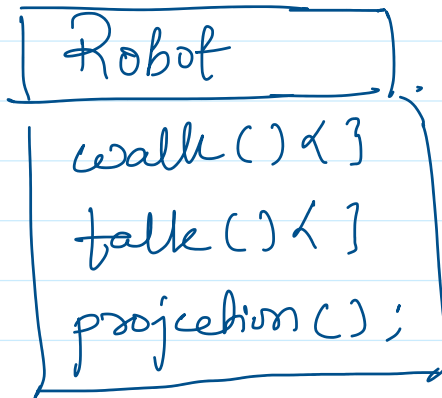
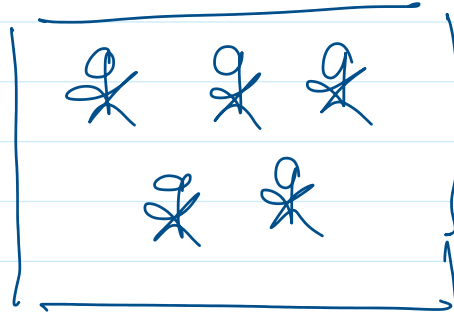
You have to extract those parts from your application which is static. and store it different class.

↓  
This is important b/c the change in dynamic part will not affect the static part.

# Strategy Design Pattern.

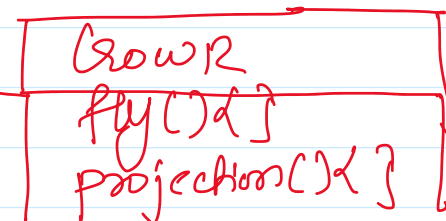
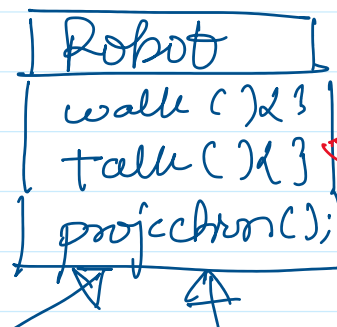
Let's create a Application in which you have to simulate the robots.

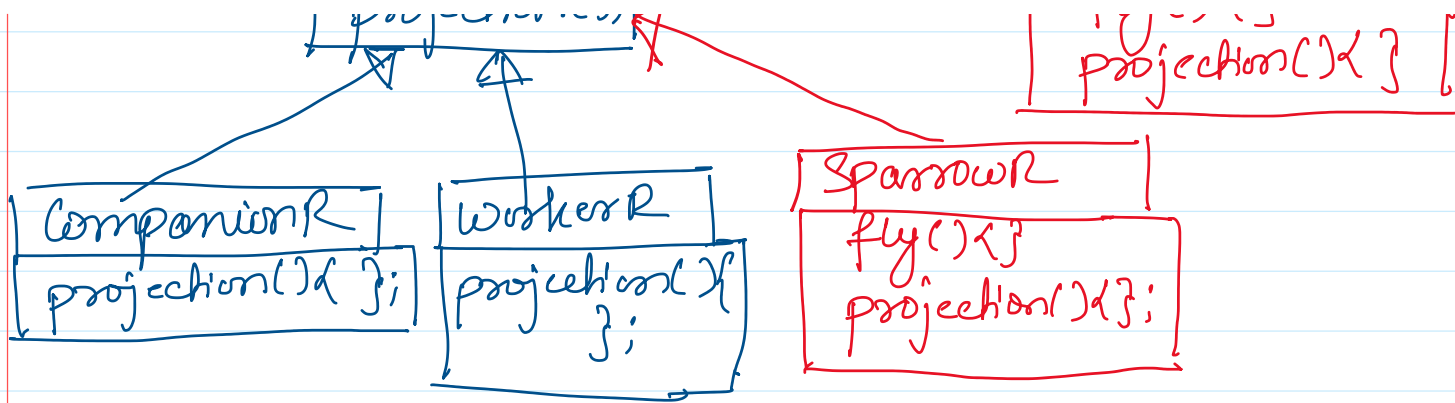
App →



→ In this design Pattern you will understand Inheritance is a very bad thing. — let's see why... and what we should do to solve it.

for now we have 2 robots — "Companion Robot" & "Worker Robot" whose walk() & talk() is same as "Robot" but projection() they will override.





Now suppose in future new Robot came "SparrowR" which can fly so we will simply add fly() method in "SparrowR" class.

Now again in future new robot comes i.e "CrowR" which can also fly and it has same implementation as one in "SparrowR"

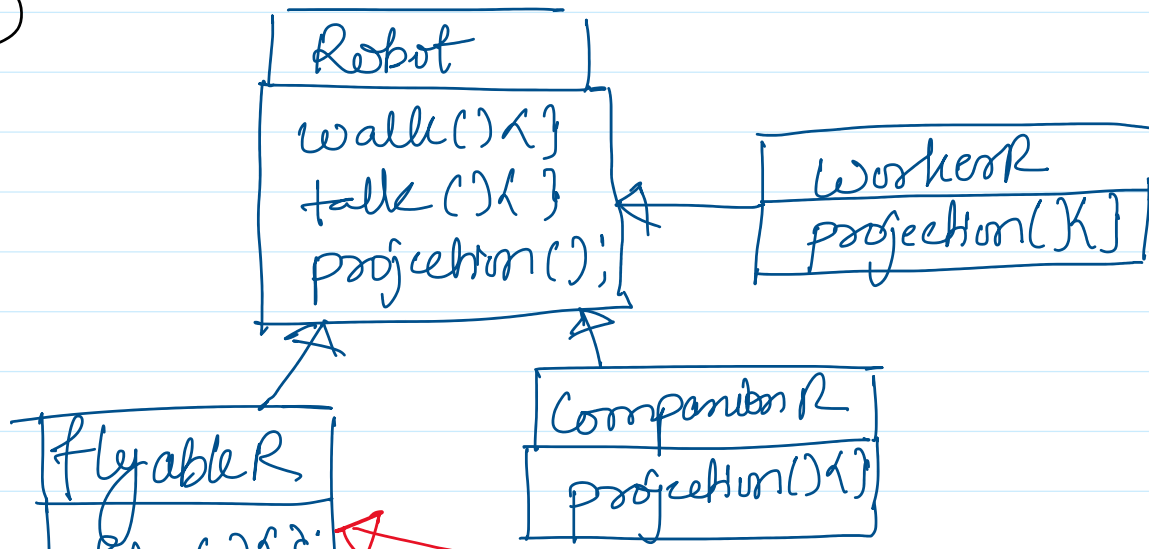
So here we are breaking a principle called 'DRY'

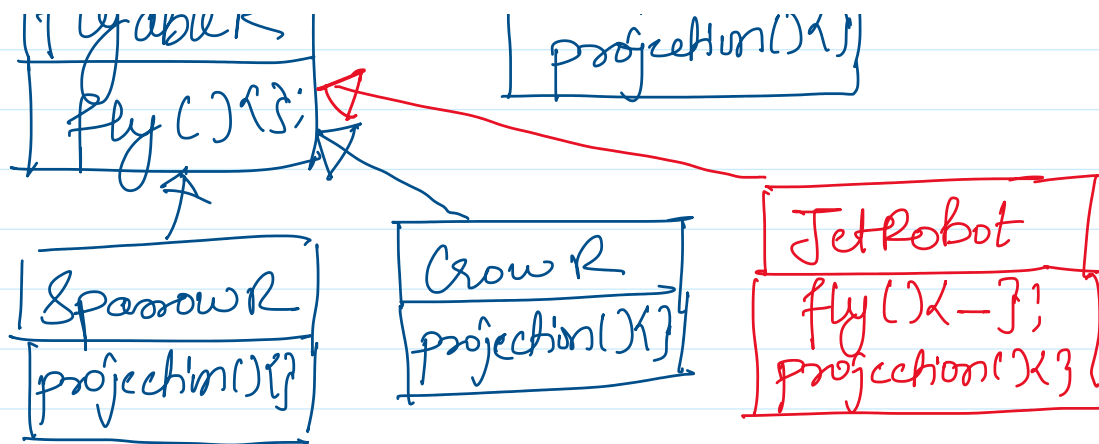
→ Don't Repeat Yourself.

Solution:-

- ① If we add fly(); in "Robot" class then all Robots will fly which is wrong. — Wrong X

②



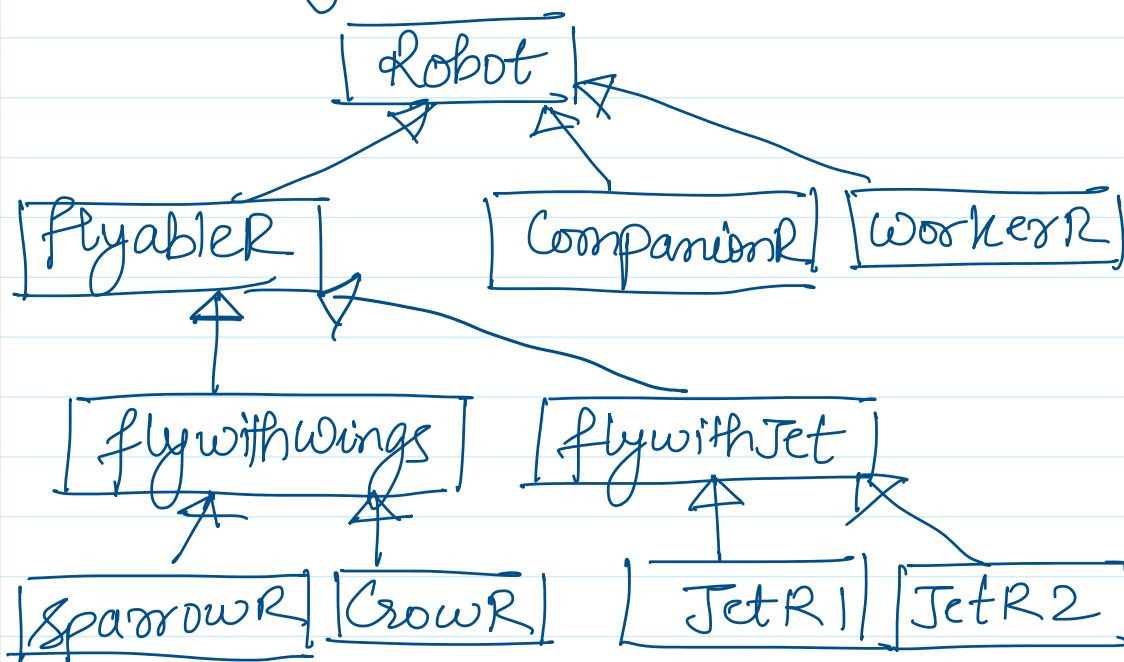


Now suppose new Robot comes "JetRobot" which fly differently then we can simply add above and override the fly() method

Now again same problem i.e new "JetRobot2" comes then again in that you will write fly() method.

So again we are violating 'DRY' principal.

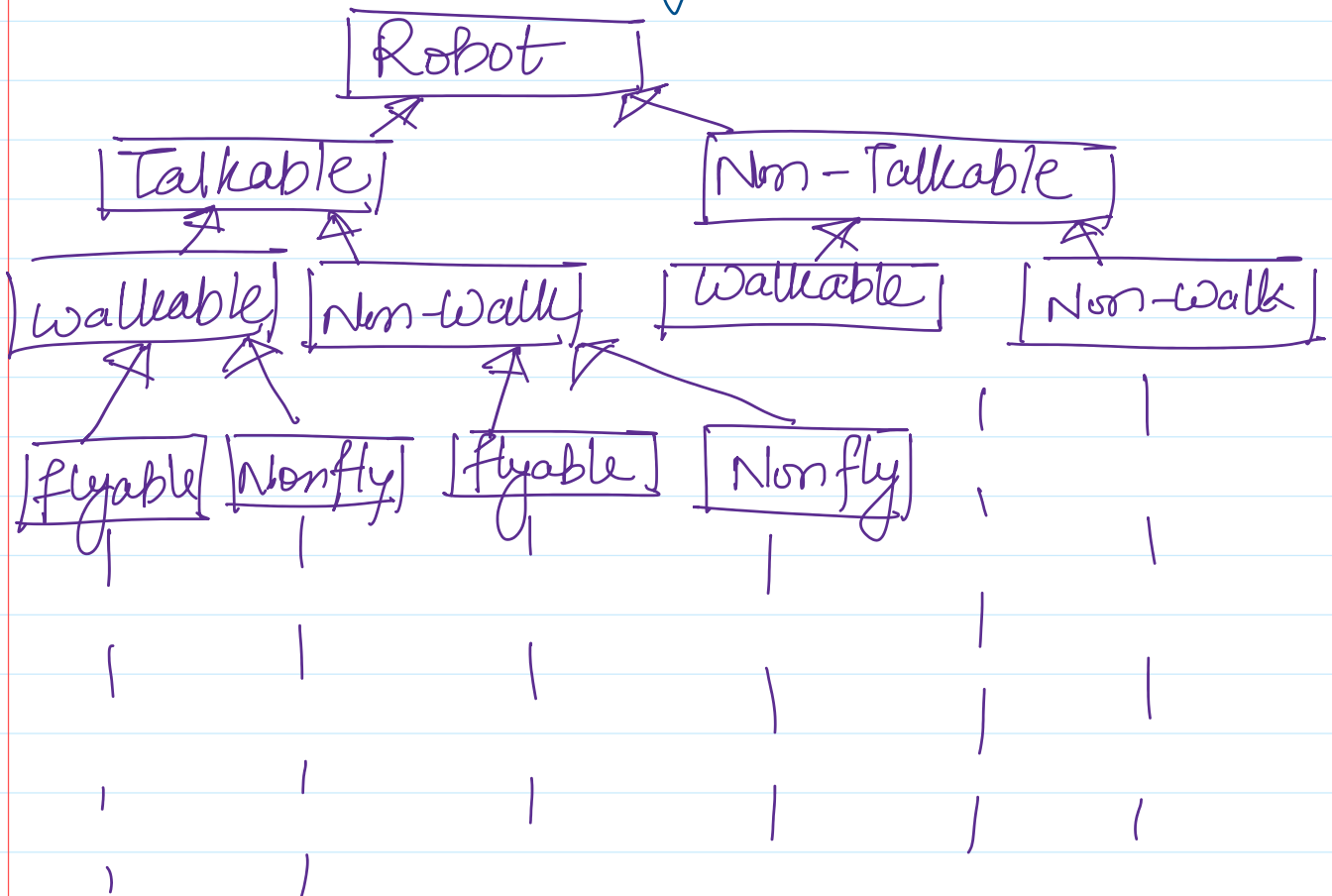
Now again we have to increase our hierarchy. like this:-



So this hierarchy will keep on nothing complicated as new.

So this hierarchy will keep on getting complicated as new types of Robot come.

At last it will look like this, if new robot come who can't talk / can't walk / can't fly :-



Note - Solution of Inheritance is not more Inheritance.

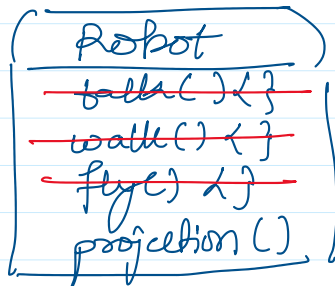
Problems with Inheritance

- Code Re-use
- To add new feature a lot of changes were required
- Breaking OCP.

# Definition of Strategy Design Pattern

Defines a family of algorithms, put them into separate classes so that they can be changed at run time.

let's use it in our Robot Example.  
let's separate out things - Static & Dynamic Part

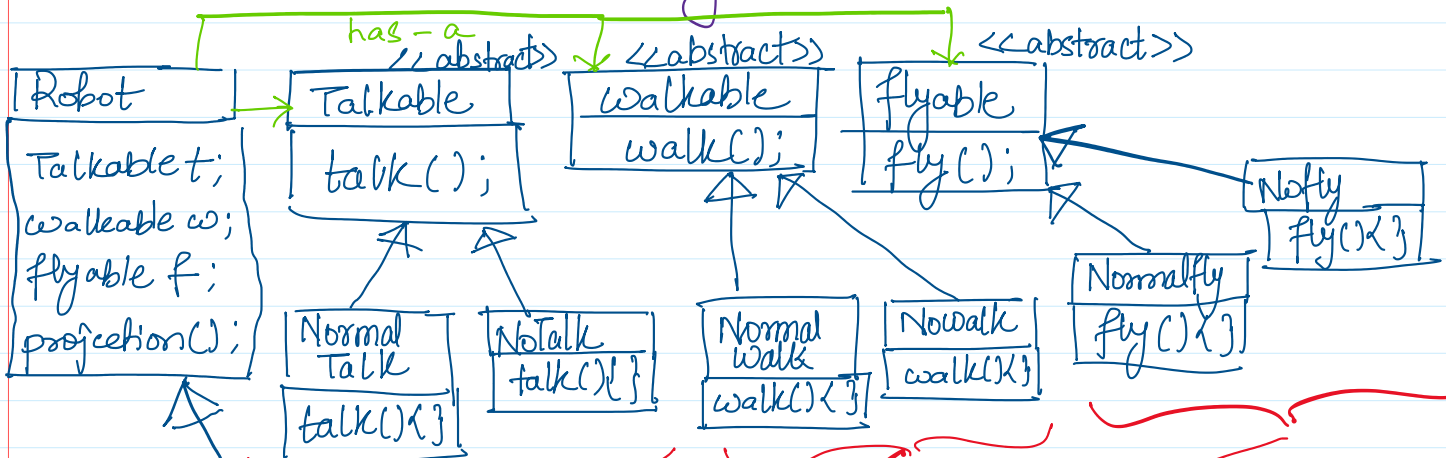


talk(), walk(), fly() can change so they are dynamic separate them and projection() will be implemented by every robot so it static.

talk()  
walk()  
fly()

family of algorithms  
↓  
separate them in different class

Read the Definition again



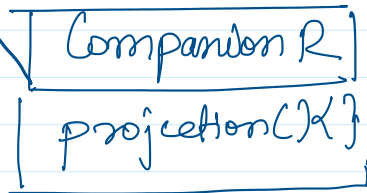
This is called "Client"

called "Strategy"

This is called "Client"  
b/c it uses all Algorithms

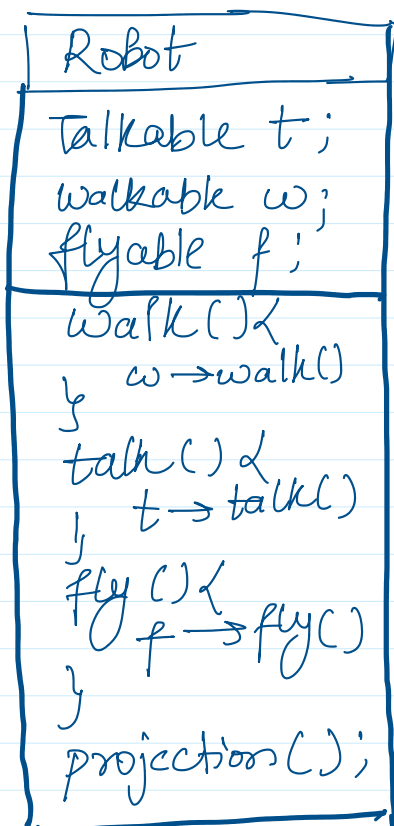
called "Strategy"  
So, there are 3 strategy.

Let's create new robot which can talk, walk but don't fly



```
Robot * robot = new CompanionR (
    new NormalTalk();
    new NormalWalk();
    new NoFly();
);
```

Robot Class will have walk(), talk() & fly() method which will just delegate its work



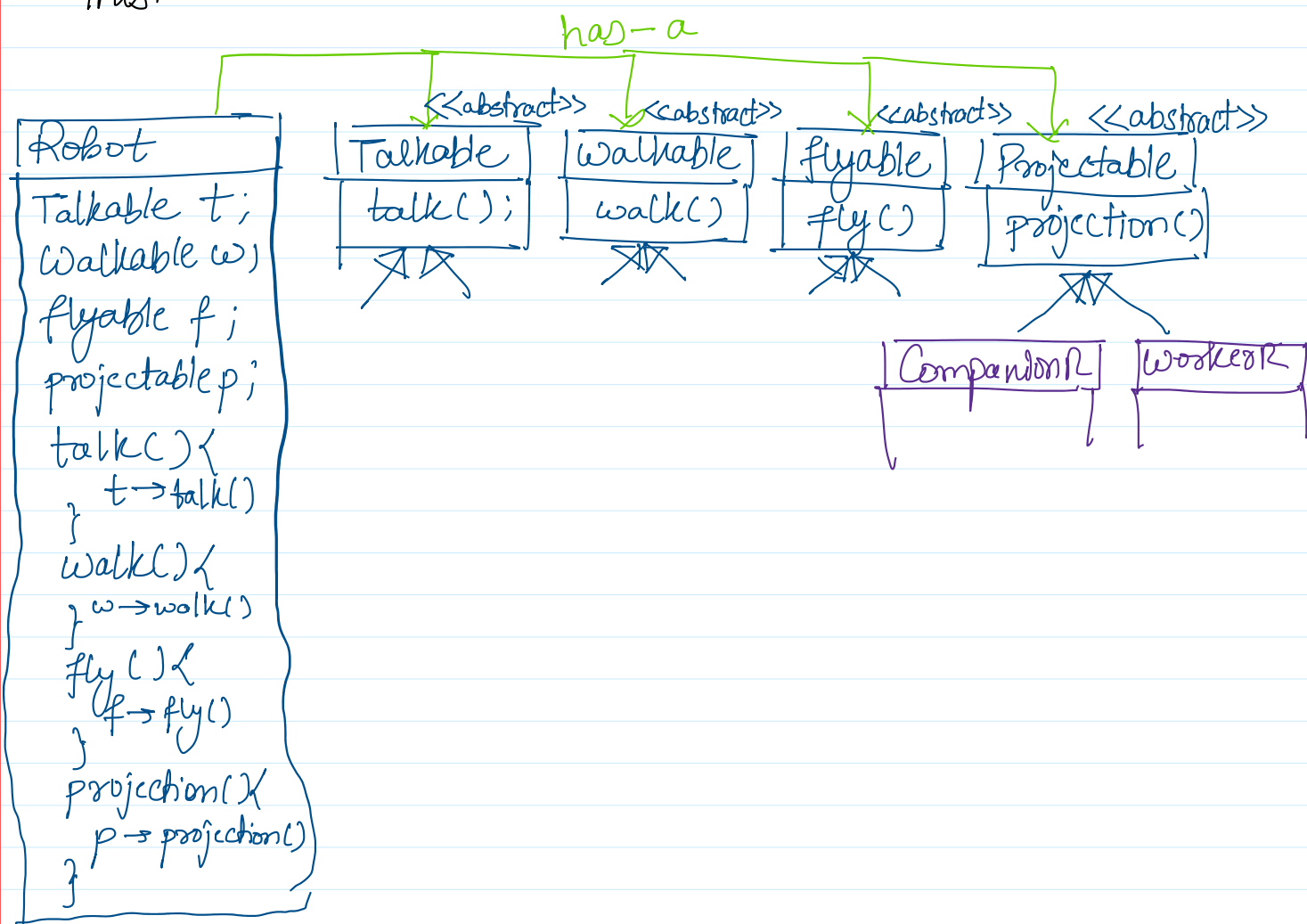
So, when you want to add new Robot, you can simply add new class. → So it is supporting OCP also

class `Gun` ...  
→ So it is supporting OCP also ✓

Code → See Notes...

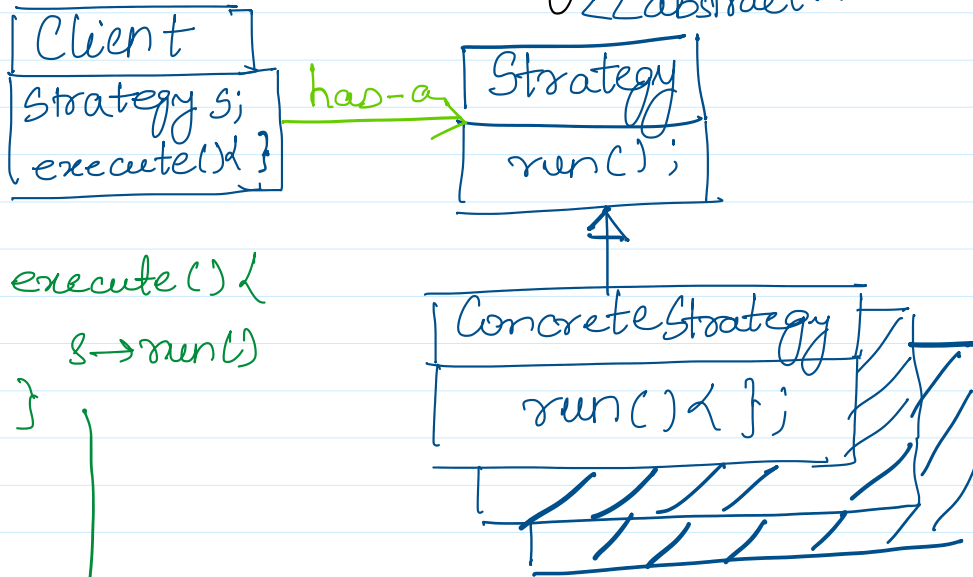
Still when new Robot comes we have to create child class of "Robot" class like "CompanionR", "WorkerR" etc.

So again we can solve this by moving "projection()" to new abstract class like this:-



Robot \* r = new Robot ( \_\_\_\_\_  
\_\_\_\_\_  
\_\_\_\_\_  
\_\_\_\_\_  
);

## Standard UML Diagram.



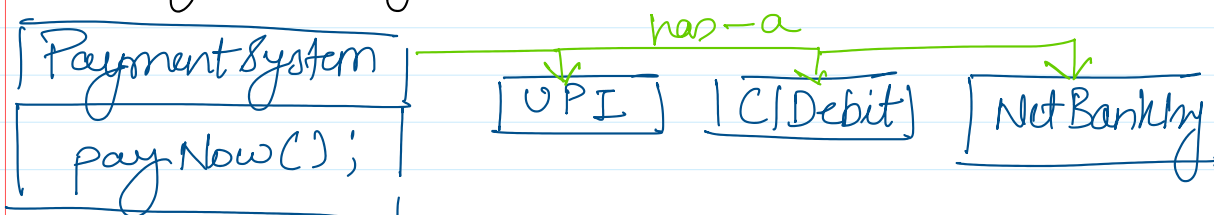
`execute() {`  
`s → run()`  
`}`

Depends on which ConcreteStrategy Class you passed → that `run()` will get executed

↓  
Dynamic Polymorphism

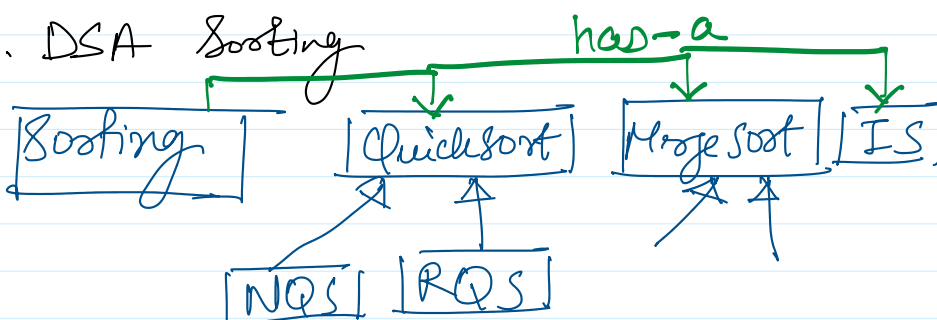
## Different Examples:-

### 1. Payment System.



Depends upon which object you pass, that type of payment will happen.

### 2. DSA Sorting



NQS RQS

/ /

## Conclusion:-

- Encapsulate what varies & keep it separate from what remains same.
- solution to inheritance is not more inheritance
- Composition should be favoured over inheritance
-